

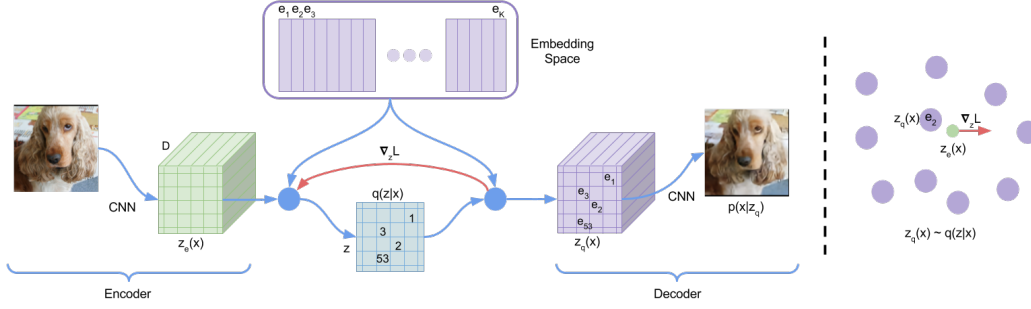


Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2 . The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

During forward computation the nearest embedding $z_q(x)$ (equation 2) is passed to the decoder, and during the backwards pass the gradient $\nabla_z L$ is passed unaltered to the encoder. Since the output representation of the encoder and the input to the decoder share the same D dimensional space, the gradients contain useful information for how the encoder has to change its output to lower the reconstruction loss.

As seen on Figure 1 (right), the gradient can push the encoder’s output to be discretised differently in the next forward pass, because the assignment in equation 1 will be different.

Equation 3 specifies the overall loss function. It has three components that are used to train different parts of VQ-VAE. The first term is the reconstruction loss (or the data term) which optimizes the decoder and the encoder (through the estimator explained above). Due to the straight-through gradient estimation of mapping from $z_e(x)$ to $z_q(x)$, the embeddings e_i receive no gradients from the reconstruction loss $\log p(z|z_q(x))$. Therefore, in order to learn the embedding space, we use one of the simplest dictionary learning algorithms, Vector Quantisation (VQ). The VQ objective uses the l_2 error to move the embedding vectors e_i towards the encoder outputs $z_e(x)$ as shown in the second term of equation 3. Because this loss term is only used for updating the dictionary, one can alternatively also update the dictionary items as function of moving averages of $z_e(x)$ (not used for the experiments in this work). For more details see Appendix A.1.

Finally, since the volume of the embedding space is dimensionless, it can grow arbitrarily if the embeddings e_i do not train as fast as the encoder parameters. To make sure the encoder commits to an embedding and its output does not grow, we add a commitment loss, the third term in equation 3. Thus, the total training objective becomes:

$$L = \log p(x|z_q(x)) + \| \text{sg}[z_e(x)] - e \|_2^2 + \beta \| z_e(x) - \text{sg}[e] \|_2^2, \quad (3)$$

where sg stands for the stopgradient operator that is defined as identity at forward computation time and has zero partial derivatives, thus effectively constraining its operand to be a non-updated constant. The decoder optimises the first loss term only, the encoder optimises the first and the last loss terms, and the embeddings are optimised by the middle loss term. We found the resulting algorithm to be quite robust to β , as the results did not vary for values of β ranging from 0.1 to 2.0. We use $\beta = 0.25$ in all our experiments, although in general this would depend on the scale of reconstruction loss. Since we assume a uniform prior for z , the KL term that usually appears in the ELBO is constant w.r.t. the encoder parameters and can thus be ignored for training.

In our experiments we define N discrete latents (e.g., we use a field of 32×32 latents for ImageNet, or $8 \times 8 \times 10$ for CIFAR10). The resulting loss L is identical, except that we get an average over N terms for k -means and commitment loss – one for each latent.

The log-likelihood of the complete model $\log p(x)$ can be evaluated as follows:

$$\log p(x) = \log \sum_k p(x|z_k) p(z_k),$$

Because the decoder $p(x|z)$ is trained with $z = z_q(x)$ from MAP-inference, the decoder should not allocate any probability mass to $p(x|z)$ for $z \neq z_q(x)$ once it has fully converged. Thus, we can write

$\log p(x) \approx \log p(x|z_q(x))p(z_q(x))$. We empirically evaluate this approximation in section 4. From Jensen’s inequality, we also can write $\log p(x) \geq \log p(x|z_q(x))p(z_q(x))$.

3.3 Prior

The prior distribution over the discrete latents $p(z)$ is a categorical distribution, and can be made autoregressive by depending on other z in the feature map. Whilst training the VQ-VAE, the prior is kept constant and uniform. After training, we fit an autoregressive distribution over z , $p(z)$, so that we can generate x via ancestral sampling. We use a PixelCNN over the discrete latents for images, and a WaveNet for raw audio. Training the prior and the VQ-VAE jointly, which could strengthen our results, is left as future research.

4 Experiments

4.1 Comparison with continuous variables

As a first experiment we compare VQ-VAE with normal VAEs (with continuous variables), as well as VIMCO [28] with independent Gaussian or categorical priors. We train these models using the same standard VAE architecture on CIFAR10, while varying the latent capacity (number of continuous or discrete latent variables, as well as the dimensionality of the discrete space K). The encoder consists of 2 strided convolutional layers with stride 2 and window size 4×4 , followed by two residual 3×3 blocks (implemented as ReLU, 3×3 conv, ReLU, 1×1 conv), all having 256 hidden units. The decoder similarly has two residual 3×3 blocks, followed by two transposed convolutions with stride 2 and window size 4×4 . We use the ADAM optimiser [21] with learning rate $2e-4$ and evaluate the performance after 250,000 steps with batch-size 128. For VIMCO we use 50 samples in the multi-sample training objective.

The VAE, VQ-VAE and VIMCO models obtain **4.51** bits/dim, **4.67** bits/dim and **5.14** respectively. All reported likelihoods are lower bounds. Our numbers for the continuous VAE are comparable to those reported for a Deep convolutional VAE: **4.54** bits/dim [13] on this dataset.

Our model is the first among those using discrete latent variables which challenges the performance of continuous VAEs. Thus, we get very good reconstructions like regular VAEs provide, with the compressed representation that symbolic representations provide. A few interesting characteristics, implications and applications of the VQ-VAEs that we train is shown in the next subsections.

4.2 Images

Images contain a lot of redundant information as most of the pixels are correlated and noisy, therefore learning models at the pixel level could be wasteful.

In this experiment we show that we can model $x = 128 \times 128 \times 3$ images by compressing them to a $z = 32 \times 32 \times 1$ discrete space (with $K=512$) via a purely deconvolutional $p(x|z)$. So a reduction of $\frac{128 \times 128 \times 3 \times 8}{32 \times 32 \times 9} \approx 42.6$ in bits. We model images by learning a powerful prior (PixelCNN) over z . This allows to not only greatly speed up training and sampling, but also to use the PixelCNNs capacity to capture the global structure instead of the low-level statistics of images.

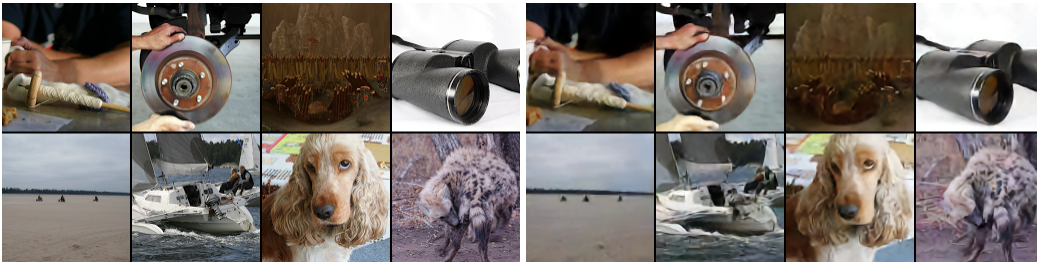


Figure 2: Left: ImageNet $128 \times 128 \times 3$ images, right: reconstructions from a VQ-VAE with a $32 \times 32 \times 1$ latent space, with $K=512$.

Reconstructions from the $32 \times 32 \times 1$ space with discrete latents are shown in Figure 2. Even considering that we greatly reduce the dimensionality with discrete encoding, the reconstructions look only slightly blurrier than the originals. It would be possible to use a more perceptual loss function than MSE over pixels here (e.g., a GAN [12]), but we leave that as future work.

Next, we train a PixelCNN prior on the discretised $32 \times 32 \times 1$ latent space. As we only have 1 channel (not 3 as with colours), we only have to use spatial masking in the PixelCNN. The capacity of the PixelCNN we used was similar to those used by the authors of the PixelCNN paper [38].

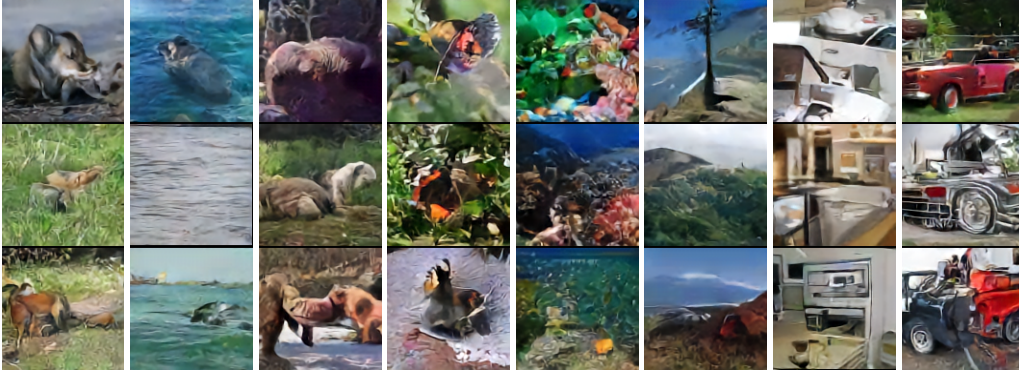


Figure 3: Samples (128×128) from a VQ-VAE with a PixelCNN prior trained on ImageNet images. From left to right: kit fox, gray whale, brown bear, admiral (butterfly), coral reef, alp, microwave, pickup.

Samples drawn from the PixelCNN were mapped to pixel-space with the decoder of the VQ-VAE and can be seen in Figure 3.

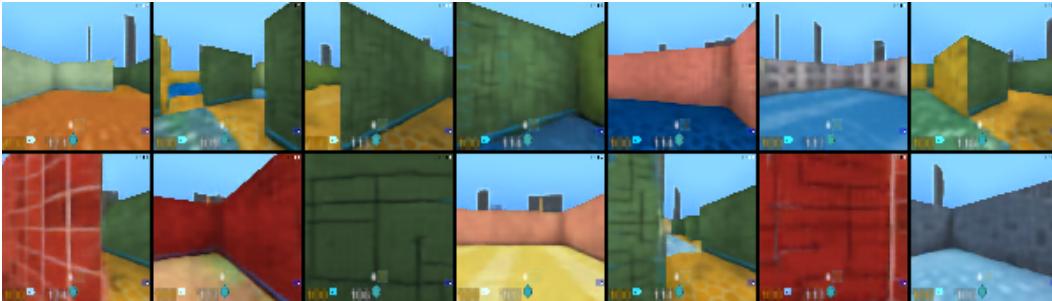


Figure 4: Samples (128×128) from a VQ-VAE with a PixelCNN prior trained on frames captured from DeepMind Lab.

We also repeat the same experiment for $84 \times 84 \times 3$ frames drawn from the DeepMind Lab environment [2]. The reconstructions looked nearly identical to their originals. Samples drawn from the PixelCNN prior trained on the $21 \times 21 \times 1$ latent space and decoded to the pixel space using a deconvolutional model decoder can be seen in Figure 4.

Finally, we train a second VQ-VAE with a *PixelCNN decoder* on top of the $21 \times 21 \times 1$ latent space from the first VQ-VAE on DM-LAB frames. This setup typically breaks VAEs as they suffer from "posterior collapse", i.e., the latents are ignored as the decoder is powerful enough to model x perfectly. Our model however does not suffer from this, and the latents are meaningfully used. We use only three latent variables (each with $K=512$ and their own embedding space e) at the second stage for modelling the whole image and as such the model cannot reconstruct the image perfectly – which is consequence of compressing the image onto 3×9 bits, i.e. less than a float32. Reconstructions sampled from the discretised global code can be seen in Figure 5.